RetinAI

Fondements Mathématiques et Algorithmiques du Pipeline

Analyse détaillée avec interprétations intuitives pour étudiants en L2–L3 / CPGE

Pipeline ML · TensorFlow · Computer Vision

Mars 2026

Rappels de cours · Interprétation intuitive · Remarques · Points de vigilance

Table des matières

1 Introduction et prérequis **2**

2 Prétraitement des images rétiniennes **2**
2.1 Représentation mathématique d'une image . . . 2
2.2 Convolution et filtrage . . . 2
2.3 Normalisation Ben-Graham . . . 3
2.4 CLAHE . . . 4

3 Réseaux de neurones convolutifs **4**
3.1 Le neurone artificiel . . . 4
3.2 Convolution 2D dans les CNN . . . 5
3.3 Fonctions d'activation . . . 5
3.4 Batch Normalization . . . 6
3.5 EfficientNetV2 et Squeeze-and-Excitation . . . 6
3.6 Attention spatiale (CBAM) . . . 6
3.7 Transfer Learning . . . 7

4 Fonctions de perte **7**
4.1 Cross-Entropy . . . 7
4.2 Focal Loss . . . 7
4.3 Label Smoothing . . . 8
4.4 Quadratic Weighted Kappa (QWK) Loss . . . 8

5 Augmentation des données **8**
5.1 Augmentations géométriques et le groupe diédral D_4 . . . 9
5.2 MixUp . . . 9
5.3 CutMix . . . 9

6 Optimisation de l'entraînement **9**
6.1 Descente de gradient . . . 9
6.2 AdamW . . . 10

7 Optimisation à l'inférence **10**
7.1 Pruning . . . 10
7.2 Quantification . . . 10
7.3 Test-Time Augmentation . . . 11

8 Évaluation et métriques **11**
8.1 Précision, rappel, F1 . . . 11
8.2 AUC-ROC . . . 11
8.3 ECE (Expected Calibration Error) . . . 11

9 Explicabilité : Grad-CAM **11**

10 Récapitulatif des notations **12**

1 Introduction et prérequis

Ce document s'adresse aux étudiants de **L2–L3 mathématiques** ou de **CPGE (MP/PC)** qui souhaitent comprendre les fondements mathématiques du deep learning appliqué à la vision par ordinateur. Les prérequis sont :

- **Algèbre linéaire** : espaces vectoriels, produit matriciel, valeurs propres, produit scalaire.
- **Analyse** : dérivées partielles, gradient, règle de chaîne, suites, convergence.
- **Probabilités** : variables aléatoires, espérance, variance, loi de Bayes, loi Beta.
- **Analyse de Fourier (utile)** : décomposition basse/haute fréquence, convolution.

Chaque section commence par le contexte intuitif, puis développe la formalisation mathématique.

2 Prétraitement des images rétiniennes

2.1 Représentation mathématique d'une image

Image numérique

Une image couleur est une fonction $I : \{1, \dots, H\} \times \{1, \dots, W\} \to \{0, \dots, 255\}^3$ qui associe à chaque pixel (i, j) un triplet (R, G, B). On la représente comme un tenseur $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$.

Rappel : tenseur

Un tenseur d'ordre 3 est simplement une « matrice à 3 indices ». En L2, vous manipulez des matrices $A \in \mathbb{R}^{m \times n}$ (ordre 2). Ici, l'image a 3 dimensions : hauteur H, largeur W, et 3 canaux de couleur. On note $I_{i,j,c}$ l'intensité du pixel (i, j) sur le canal c.

Analogie avec l'analyse fonctionnelle

Une image en niveaux de gris est une fonction définie sur un domaine discret $\{1, \dots, H\} \times \{1, \dots, W\}$ à valeurs dans $[0, 255]$, exactement comme les fonctions $f : \mathbb{R}^2 \to \mathbb{R}$ que vous étudiez en analyse. Le traitement d'images consiste à appliquer des *opérateurs* (linéaires ou non) à ces fonctions.

2.2 Convolution et filtrage

Rappel : produit de convolution (L2 Analyse)

En analyse, le produit de convolution de f et g est :

$$(f * g)(x) = \int_{-\infty}^{+\infty} f(t)\, g(x - t)\, dt$$

C'est un opérateur **linéaire** et **invariant par translation**. En traitement d'images, on

passe au cas discret et 2D :

$$(I * K)(i,j) = \sum_m \sum_n I(i-m,\, j-n)\, K(m,n)$$

Ce que fait une convolution sur une image

La convolution réalise une **moyenne pondérée locale** : pour calculer la nouvelle valeur du pixel (i,j), on prend une combinaison linéaire de ses voisins, avec des poids donnés par le noyau K. Le choix de K détermine l'effet :

- K = matrice constante $\frac{1}{n^2}$ $\rightarrow$ flou (moyenne locale).
- K avec valeurs positives au centre et négatives autour $\rightarrow$ détection de contours.
- K gaussien $\rightarrow$ flou doux (suppression des hautes fréquences).

2.3 Normalisation Ben-Graham

Le noyau gaussien 2D est :

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right) \tag{1}$$

Rappel : la gaussienne 2D

Vous connaissez la loi $\mathcal{N}(0,\sigma^2)$. Le noyau G_σ est sa version 2D. C'est le *seul* noyau qui est à la fois :

- **Séparable** : $G_\sigma(x,y) = g_\sigma(x) \cdot g_\sigma(y)$ (produit de deux 1D).
- **Isotrope** : invariant par rotation (ne dépend que de $\sqrt{x^2+y^2}$).
- **Stable par convolution** : $G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2+\sigma_2^2}}$.

La transformation Ben-Graham soustrait le flou gaussien :

$$\boxed{I_{\text{norm}} = \alpha(I - I * G_\sigma) + \beta} \tag{2}$$

avec $\alpha = 4$ et $\beta = 128$.

Interprétation : filtre passe-haut

Décomposons l'image en deux composantes :

$$I = \underbrace{I * G_\sigma}_{\text{basse fréquence (éclairage)}} + \underbrace{(I - I * G_\sigma)}_{\text{haute fréquence (détails)}}$$

C'est comme projeter un vecteur sur un sous-espace et son orthogonal. La formule ne garde que les hautes fréquences (vaisseaux, lésions) et supprime l'illumination non-uniforme.
Lien avec Fourier : la transformée de Fourier d'une gaussienne est une gaussienne. Un

noyau large (σ grand) tue les hautes fréquences. Soustraire $I * G_\sigma$ revient à appliquer un filtre passe-haut.

2.4 CLAHE

Rappel : fonction de répartition (L2 Probabilités)

Pour une v.a. X, la CDF est $F_X(t) = P(X \leq t)$. L'histogramme d'une image est l'analogue empirique : $h(k) = \#\{(i,j) : I(i,j) = k\}$, et :

$$\mathrm{CDF}(k) = \sum_{i=0}^{k} h(i)$$

L'égalisation d'histogramme applique :

$$T(k) = \left\lfloor 255 \cdot \frac{\mathrm{CDF}(k) - \mathrm{CDF}_{\min}}{H \cdot W - \mathrm{CDF}_{\min}} \right\rfloor \tag{3}$$

Le lemme de la CDF inverse

En probabilités, si $U \sim \mathrm{Uniforme}([0,1])$, alors $F_X^{-1}(U) \sim X$. Réciproquement, $F_X(X) \sim \mathrm{Uniforme}([0,1])$. L'égalisation d'histogramme fait *exactement* cela : on applique la CDF empirique à chaque pixel, transformant la distribution des intensités en distribution quasi-uniforme. C'est la **transformation quantile** des statisticiens.

Le CLAHE ajoute deux améliorations : (1) l'égalisation est **locale** (par tuiles 8×8 avec interpolation bilinéaire), et (2) l'histogramme est **écrêté** (seuil $\tau = 2.0$) pour limiter l'amplification du bruit.

Analogie : un égaliseur audio adaptatif

En audio, un compresseur limite les pics de volume tout en amplifiant les sons faibles. Le CLAHE fait la même chose *localement* sur une image : il amplifie le contraste dans les zones sombres tout en le limitant dans les zones déjà contrastées. Le paramètre τ est le seuil de compression.

3 Réseaux de neurones convolutifs

3.1 Le neurone artificiel

Un neurone calcule : $y = \varphi(\mathbf{w}^\top \mathbf{x} + b)$

Interprétation géométrique

Sans φ, un neurone calcule un **produit scalaire** $\langle \mathbf{w}, \mathbf{x} \rangle + b$: c'est l'équation d'un hyperplan dans $\mathbb{R}^n$. Le neurone mesure de quel côté de cet hyperplan se trouve $\mathbf{x}$. La fonction φ

« décide » ensuite si le neurone s'active.
Un réseau de neurones empile ces partitions pour créer des régions de décision arbitrairement complexes. Sans non-linéarité, un réseau à L couches calcule $f(\mathbf{x}) = W_L \cdots W_1 \mathbf{x}$: une simple application linéaire (le produit de matrices est une matrice). Un réseau linéaire profond **n'est pas plus expressif** qu'une seule couche !

3.2 Convolution 2D dans les CNN

Opération de convolution 2D

Soit $\mathbf{X} \in \mathbb{R}^{H \times W \times C_{\text{in}}}$ et un filtre $\mathbf{W} \in \mathbb{R}^{k \times k \times C_{\text{in}} \times C_{\text{out}}}$:

$$Y_{i,j,f} = \sum_{c=1}^{C_{\text{in}}} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} W_{m,n,c,f} \cdot X_{i+m,\, j+n,\, c} + b_f \tag{4}$$

Lien avec les matrices de Toeplitz (L3)

En 1D, la convolution discrète s'écrit comme la multiplication par une matrice tridiagonale de Toeplitz. La convolution 2D est la version bloc-Toeplitz. La structure creuse signifie beaucoup moins de paramètres qu'une couche dense : $9 \cdot C_{\text{in}} \cdot C_{\text{out}}$ au lieu de $H^2 W^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$.

Ce que « voit » un filtre convolutif

Un filtre 3×3 regarde une fenêtre locale de 9 pixels et calcule un produit scalaire avec ses poids. Si les poids ressemblent à un motif (contour vertical par exemple), le produit scalaire est grand quand l'image locale ressemble à ce motif. C'est un **détecteur de motifs locaux**.
Partage de poids : le même filtre est appliqué à *toutes* les positions $\rightarrow$ invariance par translation.
Localité : chaque sortie ne dépend que d'un voisinage $k \times k$.

3.3 Fonctions d'activation

ReLU. $\text{ReLU}(x) = \max(0, x)$. Dérivée : $\mathbf{1}_{x>0}$ (0 ou 1), donc pas de vanishing gradient.

Swish. Utilisée dans EfficientNetV2 : $\text{Swish}(x) = \frac{x}{1+e^{-x}}$. Fonction C^∞, laisse passer un petit signal négatif.

Softmax. Pour la classification :

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{5}$$

Lien avec la mesure de Gibbs

La softmax est la distribution de Boltzmann de la physique statistique : $P(\text{état } i) \propto e^{-E_i/kT}$. C'est la distribution qui **maximise l'entropie** sous contrainte d'énergie moyenne fixée. Elle

est aussi le gradient de log-sum-exp : $\nabla_{\mathbf{z}} \log(\sum_j e^{z_j}) = \mathrm{softmax}(\mathbf{z})$.

3.4 Batch Normalization

Pour un mini-batch $\mathcal{B} = \{x_1, \ldots, x_m\}$:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y_i = \gamma\, \hat{x}_i + \beta \tag{6}$$

Interprétation statistique

L'étape $\hat{x}_i = \frac{x_i - \mu}{\sigma}$ est la **standardisation** (variable centrée réduite). En géométrie, cela recentre le nuage à l'origine et normalise sa variance à 1. Cela stabilise les activations et permet des learning rates plus élevés. Les paramètres appris γ, β permettent au réseau de « défaire » la normalisation si nécessaire.

3.5 EfficientNetV2 et Squeeze-and-Excitation

Le module SE recalibre les canaux selon leur importance :

$$\mathbf{s} = \sigma(\mathbf{W}_2\, \mathrm{ReLU}(\mathbf{W}_1\, \mathrm{GAP}(\mathbf{F}))) \in \mathbb{R}^C \tag{7}$$

$$\widetilde{\mathbf{F}}_c = s_c \cdot \mathbf{F}_c \tag{8}$$

Analogie : une ACP adaptative et apprise

L'ACP (L3) identifie les directions de variance maximale. Le module SE fait quelque chose de similaire mais **adaptatif** : pour chaque image, il identifie quels canaux sont les plus informatifs ($s_c > 0.5$) et atténue les canaux bruités ($s_c < 0.5$). Le GAP résume chaque canal en un scalaire (sa moyenne spatiale), puis le MLP ($C \to C/r \to C$ avec $r = 16$) agit comme un goulot d'étranglement qui force une représentation comprimée des relations inter-canaux.

3.6 Attention spatiale (CBAM)

Attention par canal : $\mathbf{M}_c(\mathbf{F}) = \sigma(\mathrm{MLP}(\mathrm{AvgPool}(\mathbf{F})) + \mathrm{MLP}(\mathrm{MaxPool}(\mathbf{F})))$
Attention spatiale ($\mathbf{F}' = \mathbf{M}_c \odot \mathbf{F}$) : $\mathbf{M}_s(\mathbf{F}') = \sigma\big(f^{7\times 7}([\mathrm{AvgPool}_c(\mathbf{F}');\ \mathrm{MaxPool}_c(\mathbf{F}')])\big)$
Sortie : $\boxed{\widetilde{\mathbf{F}} = \mathbf{M}_s(\mathbf{F}') \odot \mathbf{F}'}$

La carte de saillance comme une « loupe adaptative »

$\mathbf{M}_s \in [0,1]^{H\times W}$ dit, pour chaque position, « à quel point cette zone est intéressante ». En imagerie rétinienne, les micro-anévrismes (quelques pixels) sont les régions discriminantes. Sans attention, le réseau traite uniformément les 512^2 pixels. Avec $\mathbf{M}_s$, il apprend à concentrer ses ressources sur les zones de lésions. C'est comme donner au réseau une loupe qui se déplace automatiquement.

3.7 Transfer Learning

Pourquoi le transfert marche : hiérarchie des features

Les couches basses d'un CNN apprennent des features universelles (contours, textures). Les couches hautes sont spécifiques à la tâche.

Analogie d'algèbre linéaire : les premières couches construisent une « bonne base » de l'espace des images naturelles. Il suffit d'ajuster les dernières couches (les coordonnées dans cette base) pour la nouvelle tâche.

Phase 1 (warmup) : backbone gelé ($\nabla_{\theta_{\text{backbone}}} = 0$), on n'entraîne que le classifieur.

Phase 2 (fine-tuning) : on dégèle les dernières couches avec un petit learning rate.

Pourquoi deux phases ?

Si l'on dégèle immédiatement, le classifieur (initialisé aléatoirement) produit des gradients quasi-aléatoires qui détruisent les features apprises. C'est comme si un élève qui ne comprend rien corrigeait le professeur.

4 Fonctions de perte

4.1 Cross-Entropy

Rappel : divergence de Kullback-Leibler (L3)

$$D_{\text{KL}}(p\|q) = \sum_k p_k \log \frac{p_k}{q_k} = \underbrace{-\sum_k p_k \log q_k}_{\text{cross-entropy}} - \underbrace{(-\sum_k p_k \log p_k)}_{H(p)}$$

Comme $H(p)$ est constant, minimiser la CE revient à minimiser $D_{\text{KL}}(p\|q)$.

$$\mathcal{L}_{\text{CE}} = -\sum_{k=1}^{K} y_k \log(\hat{y}_k) \xrightarrow{\text{one-hot}} -\log(p_t) \tag{9}$$

La CE comme surprise

$-\log(p_t)$ mesure la « surprise » du modèle face à la bonne réponse. Si $p_t = 0.99$, loss ≈ 0.01. Si $p_t = 0.01$, loss ≈ 4.6. Minimiser la CE = minimiser la surprise moyenne.

Vue géométrique : dans le simplexe $\Delta^{K-1} = \{q \in \mathbb{R}^K : q_i \geq 0, \sum q_i = 1\}$, la CE mesure une « distance » (non symétrique) entre $\hat{\mathbf{y}}$ et le sommet de la classe correcte.

4.2 Focal Loss

$$\boxed{\mathcal{L}_{\text{FL}}(p_t) = -\alpha_t \cdot (1 - p_t)^\gamma \cdot \log(p_t)} \tag{10}$$

Dissection du facteur focal

Cas	p_t	$(1-p_t)^2$	Contribution
Bien classé	0.95	0.0025	$\times 1$
Moyen	0.70	0.09	$\times 36$
Mal classé	0.20	0.64	$\times 256$

Avec $\gamma = 2$, le modèle passe 256 fois plus d'« énergie » sur les cas difficiles. C'est de la pédagogie différenciée appliquée aux réseaux de neurones !
La pondération $\alpha_c = \frac{N}{K \cdot N_c}$ compense le déséquilibre des classes : le grade 4 (200 images sur 10 000) reçoit un poids de 10, le grade 0 (7300 images) un poids de 0.27.

4.3 Label Smoothing

$$y'_k = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{K} & \text{si } k = c \\ \frac{\varepsilon}{K} & \text{sinon} \end{cases} \tag{11}$$

Pourquoi ne pas viser la perfection ?

Avec un one-hot dur, le modèle vise $\hat{y}_c \to 1$, soit $z_c \to +\infty$. Cela crée un modèle **surconfiant**. Le label smoothing dit : « 96% de chance que ce soit la classe c, mais 4% d'incertitude ». En termes d'entropie, on augmente $H(\mathbf{y})$ de 0 (certitude totale) à $H(\mathbf{y}') > 0$. C'est une **régularisation entropique**.

4.4 Quadratic Weighted Kappa (QWK) Loss

$$\kappa = 1 - \frac{\sum_{i,j} w_{ij}\, O_{ij}}{\sum_{i,j} w_{ij}\, E_{ij}}, \quad w_{ij} = \frac{(i-j)^2}{(K-1)^2} \tag{12}$$

Le QWK comme un χ^2 pondéré

La structure $1 - \frac{\text{observé}}{\text{attendu}}$ rappelle le test du χ^2 d'indépendance (L2 Statistiques). $\kappa = 1$: accord parfait. $\kappa = 0$: pas mieux que le hasard.
La pénalité quadratique reflète la sévérité clinique : confondre grade 0 et grade 1 ($w = 1/16$) est bénin, mais confondre grade 0 et grade 4 ($w = 1$) est potentiellement mortel. C'est une distance au carré $d(i,j) = (i-j)^2$.

5 Augmentation des données

5.1 Augmentations géométriques et le groupe diédral D_4

Rappel : le groupe D_4 (L2 Algèbre)

D_4 est le groupe des isométries du carré, d'ordre 8, engendré par une rotation r de $\pi/2$ et une réflexion s : $D_4 = \langle r, s \mid r^4 = s^2 = e,\ srs = r^{-1} \rangle$. Ses 8 éléments (4 rotations + 4 réflexions) correspondent exactement aux augmentations appliquées aux images rétiniennes, qui sont invariantes par le groupe entier.

5.2 MixUp

$$\tilde{\mathbf{x}} = \lambda\, \mathbf{x}_i + (1-\lambda)\, \mathbf{x}_j, \quad \tilde{\mathbf{y}} = \lambda\, \mathbf{y}_i + (1-\lambda)\, \mathbf{y}_j, \quad \lambda \sim \mathrm{Beta}(0.2, 0.2).$$

Rappel : la loi Beta

$\mathrm{Beta}(a,b)$ a pour densité $f(x) = \frac{x^{a-1}(1-x)^{b-1}}{B(a,b)}$ sur $[0,1]$. Pour $a = b = 0.2$, la densité est en U : λ est presque toujours proche de 0 ou 1.

Interpolation dans l'espace des images

Les images sont des points dans $\mathbb{R}^{H \times W \times 3}$. MixUp crée de nouveaux points sur les **segments** reliant des paires d'images. Le modèle apprend des frontières de décision *lisses* (régularisation implicite) et ses probabilités sont mieux calibrées.

5.3 CutMix

$$\tilde{\mathbf{x}} = \mathbf{M} \odot \mathbf{x}_i + (\mathbf{1} - \mathbf{M}) \odot \mathbf{x}_j$$

MixUp vs CutMix : globalité vs localité

MixUp superpose les images en transparence (mélange global). CutMix découpe un morceau et le colle (mélange local). CutMix force le modèle à ne pas se reposer sur une seule zone : c'est un mécanisme de **robustesse à l'occultation**.

6 Optimisation de l'entraînement

6.1 Descente de gradient

Rappel : gradient (L2 Analyse)

Le gradient $\nabla f(\mathbf{x})$ pointe dans la direction de plus forte croissance. Pour minimiser : $\mathbf{x}_{t+1} = \mathbf{x}_t - \eta \nabla f(\mathbf{x}_t)$. C'est la méthode d'Euler sur l'EDO $\dot{\mathbf{x}} = -\nabla f(\mathbf{x})$ (flot de gradient). En deep learning, le gradient est calculé par **rétropropagation** (règle de chaîne).

6.2 AdamW

$$\mathbf{m}_t = \beta_1\, \mathbf{m}_{t-1} + (1-\beta_1)\, \mathbf{g}_t \tag{13}$$

$$\mathbf{v}_t = \beta_2\, \mathbf{v}_{t-1} + (1-\beta_2)\, \mathbf{g}_t^2 \tag{14}$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta\left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t}+\epsilon} + \lambda_{\text{wd}}\, \boldsymbol{\theta}_{t-1}\right) \tag{15}$$

avec $\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}$, $\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}$.

Décryptage terme par terme

1. $\mathbf{m}_t$ = moyenne mobile du gradient (momentum). Si le gradient oscille, $\mathbf{m}_t$ s'annule ; s'il est cohérent, il accumule de l'inertie. C'est une bille avec masse qui roule sur la surface de la loss.
2. $\mathbf{v}_t$ = variance du gradient. La division par $\sqrt{\hat{\mathbf{v}}_t}$ normalise le pas : les paramètres bruités reçoivent un pas plus petit. C'est un **préconditionnement diagonal**, analogue à une descente de Newton approchée.
3. Correction du biais : à t petit, $\mathbf{m}_t$ est biaisé vers 0. La division par $(1-\beta_1^t)$ corrige.
4. Weight decay $\lambda_{\text{wd}}\boldsymbol{\theta}_{t-1}$: pénalité L^2 découplée (régularisation de Tikhonov).

7 Optimisation à l'inférence

7.1 Pruning

$w'_{ij} = w_{ij} \cdot \mathbf{1}[|w_{ij}| > \tau]$

Approximation creuse

En algèbre linéaire, le théorème d'Eckart–Young donne la meilleure approximation de rang r d'une matrice via SVD tronquée. Le pruning est analogue : en mettant les petits poids à zéro, on crée une matrice **creuse** qui approxime l'originale. Les poids proches de zéro contribuent peu et peuvent être supprimés sans perte significative.

7.2 Quantification

$q = \text{round}\left(\frac{w-z}{s}\right)$, $s = \frac{w_{\max}-w_{\min}}{2^b-1}$

Discrétisation de $\mathbb{R}$ vers $\mathbb{Z}$

C'est l'erreur d'arrondi de l'analyse numérique : on remplace $w \in \mathbb{R}$ par $q \in \{0,\ldots,255\}$ via une application affine. L'erreur est bornée par $s/2 \approx 0.004$ pour des poids dans $[-1,1]$, soit $\sim 0.4\%$ d'erreur relative. La taille du modèle est divisée par 4.

7.3 Test-Time Augmentation

$$\boxed{\hat{\mathbf{y}}_{\text{TTA}} = \frac{1}{T}\sum_{t=1}^{T} f_\theta(\mathcal{A}_t(\mathbf{x}))} \tag{16}$$

Loi des grands nombres appliquée aux prédictions

Si les $T = 8$ prédictions sont des estimateurs peu biaisés, leur moyenne a une variance divisée par T. L'écart-type empirique fournit une estimation de l'**incertitude** : un cas à haute incertitude est ambigu et nécessite un examen humain.

8 Évaluation et métriques

8.1 Précision, rappel, F1

$$\text{Précision}_k = \frac{C_{kk}}{\sum_i C_{ik}}, \quad \text{Rappel}_k = \frac{C_{kk}}{\sum_j C_{kj}}, \quad F_1 = 2\,\frac{P \cdot R}{P + R} \tag{17}$$

Pourquoi la moyenne harmonique ?

Elle pénalise les valeurs déséquilibrées : $\text{HM}(0.99, 0.01) = 0.02$, alors que la moyenne arithmétique donne 0.50. Le F_1 force les deux métriques à être simultanément bonnes. Lien : inégalité AM-HM $\text{HM}(a, b) \leq \sqrt{ab} \leq \frac{a+b}{2}$.

8.2 AUC-ROC

Interprétation probabiliste

$\text{AUC} = P(f(\mathbf{x}^+) > f(\mathbf{x}^-))$: probabilité qu'un positif reçoive un score plus élevé qu'un négatif. $\text{AUC} = 0.5$: aléatoire. $\text{AUC} = 1.0$: séparation parfaite.

8.3 ECE (Expected Calibration Error)

$\text{ECE} = \sum_{b=1}^{B} \frac{|S_b|}{N} |\text{acc}(S_b) - \text{conf}(S_b)|$

Calibration

Un modèle calibré qui dit « 80% de confiance » a raison 80% du temps. Crucial en médecine : le radiologue doit pouvoir se fier au score.

9 Explicabilité : Grad-CAM

Soit A_{ij}^k les feature maps de la dernière couche convolutive et y^c le score de la classe c :

$$\alpha_k^c = \frac{1}{H'W'} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k}, \qquad \boxed{L^c_{\text{Grad-CAM}}(i,j) = \text{ReLU}\left(\sum_k \alpha_k^c \cdot A_{ij}^k\right)} \tag{18}$$

Décryptage complet

1. Le gradient $\frac{\partial y^c}{\partial A_{ij}^k}$ mesure (règle de chaîne, L1) combien le score de la classe c change quand on perturbe le pixel (i,j) de la feature map k.
2. La moyenne α_k^c est un GAP des gradients : poids d'importance global du canal k.
3. $\sum_k \alpha_k^c A_{ij}^k$ est une **projection** dans la direction de la classe c dans l'espace des features. Comme un produit scalaire $\langle \boldsymbol{\alpha}^c, \mathbf{A}_{ij} \rangle$.
4. La ReLU ne garde que les contributions positives (zones qui *augmentent* le score).

Résumé : Grad-CAM montre « où le réseau regarde » pour prendre sa décision.

10 Récapitulatif des notations

Symbole	Signification
$\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$	Image (tenseur d'ordre 3)
G_σ	Noyau gaussien 2D
$*$	Produit de convolution discret
$K = 5$	Nombre de classes
p_t	Probabilité de la classe correcte
γ	Paramètre focal
κ	Quadratic Weighted Kappa
D_4	Groupe diédral (8 éléments)
$\lambda \sim \text{Beta}(\alpha, \alpha)$	Coefficient MixUp
η	Learning rate
$\boldsymbol{\theta}$	Paramètres du modèle
$\mathbf{m}_t, \mathbf{v}_t$	Moments AdamW
A_{ij}^k	Feature map canal k, position (i,j)
α_k^c	Poids Grad-CAM
$\odot$	Produit de Hadamard
σ	Sigmoïde $\frac{1}{1+e^{-x}}$

Fin du document — Fondements mathématiques et algorithmiques